Activity Sheet – 1

Department of Electronics and Communication Engineering

Title of Experiment

Iot Sensor Data Acquisition and Visualization

Name of the Student:	**RAGHU NANDHAN**
Registration Number:	**CH.EN.U4CCE24026**
Due date of submission: 26.01.2026	**Date of Submission: 26.01.2026**

Assessment Rubrics:

Description	**Mark Allotted**	**Mark Secured**
Software required and Important definitions	10	
Program	40	
Experimental Result	30	
Viva questions	20	
Late Submission	10	
Total Marks	**100**	

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Signature of Lab in-charge

ACTIVITY - 01

Title: IoT Sensor Data Acquisition and Visualization

1. Aim: To collect sensor data from an IoT source and visualize it for further machine learning analysis.

Software / Tools Required: Python, Google Colab / Jupyter Notebook.

2. Tasks:

1. Load an IoT sensor dataset (temperature, humidity, light, or motion).
2. Explore the dataset structure and features.
3. Plot time-series graphs for each sensor parameter.
4. Identify noise, missing values, and outliers.

Outcome: Students understand real-world IoT data characteristics.

3. Program :

Task 01: Load an IoT sensor dataset (temperature, humidity, light, or motion).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

file_path = "/content/datatraining.txt"
data = pd.read_csv(file_path)

print("First 5 records:")
display(data.head())
```

Task 02: Explore the dataset structure and features.

```
print("\nDataset Info:")
data.info()

print("\nStatistical Summary:")
display(data.describe())

print("\nColumn Names:")
print(data.columns)
```

Task 03: Plot time-series graphs for each sensor parameter.

```
sns.countplot(x="Occupancy", data=data)
plt.title("Occupancy Distribution")
plt.show()

sns.countplot(x="Occupancy", hue="Light", data=data)
plt.title("Light vs Occupancy")
plt.show()

plt.figure()
data["Temperature"].hist()
plt.title("Temperature Distribution")
plt.xlabel("Temperature")
plt.ylabel("Frequency")
plt.show()
```

Task 04: Identify noise, missing values, and outliers.

[7]

```python
print("\nMissing values in each column:")
print(data.isnull().sum())

data.fillna(method='ffill', inplace=True)

print("\nMissing values after forward fill:")
print(data.isnull().sum())

data = data.replace({True: 1, False: 0})

print("\nData Types:")
print(data.dtypes)

X = data.drop("Occupancy", axis=1)
y = data["Occupancy"]

print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

print("\nFirst 5 rows of cleaned dataset:")
display(data.head())
```

4. Experimental Results:

Task 01: Load an IoT sensor dataset (temperature, humidity, light, or motion).

First 5 records:

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

Task 02: Explore the dataset structure and features.

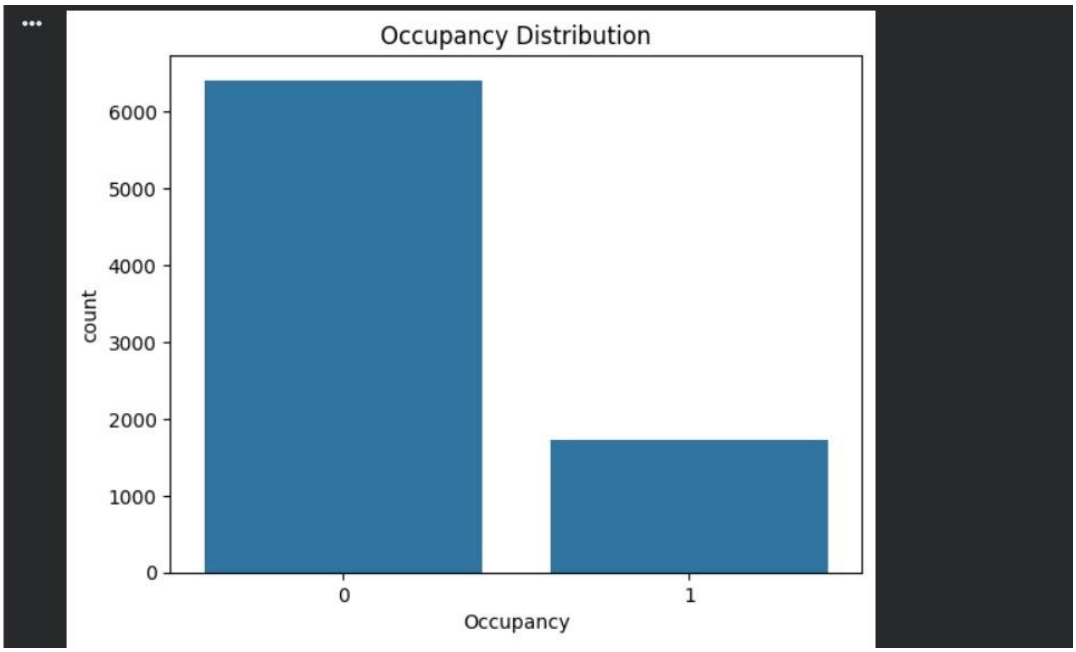
```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
Index: 8143 entries, 1 to 8143
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   date           8143 non-null   object
 1   Temperature    8143 non-null   float64
 2   Humidity       8143 non-null   float64
 3   Light          8143 non-null   float64
 4   CO2            8143 non-null   float64
 5   HumidityRatio  8143 non-null   float64
 6   Occupancy      8143 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 508.9+ KB
```

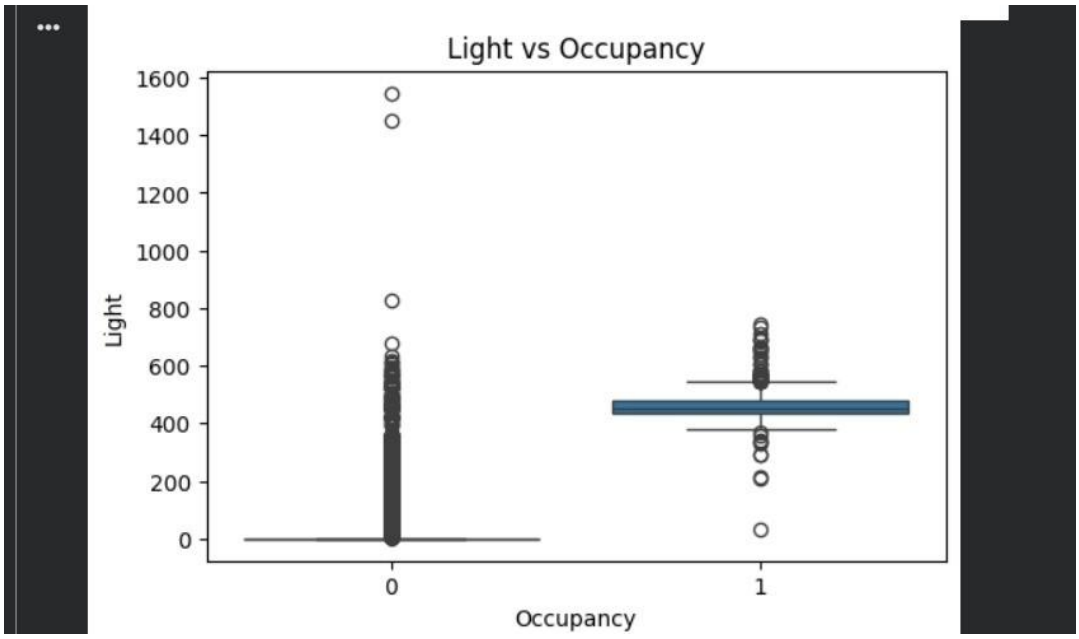
Statistical Summary:

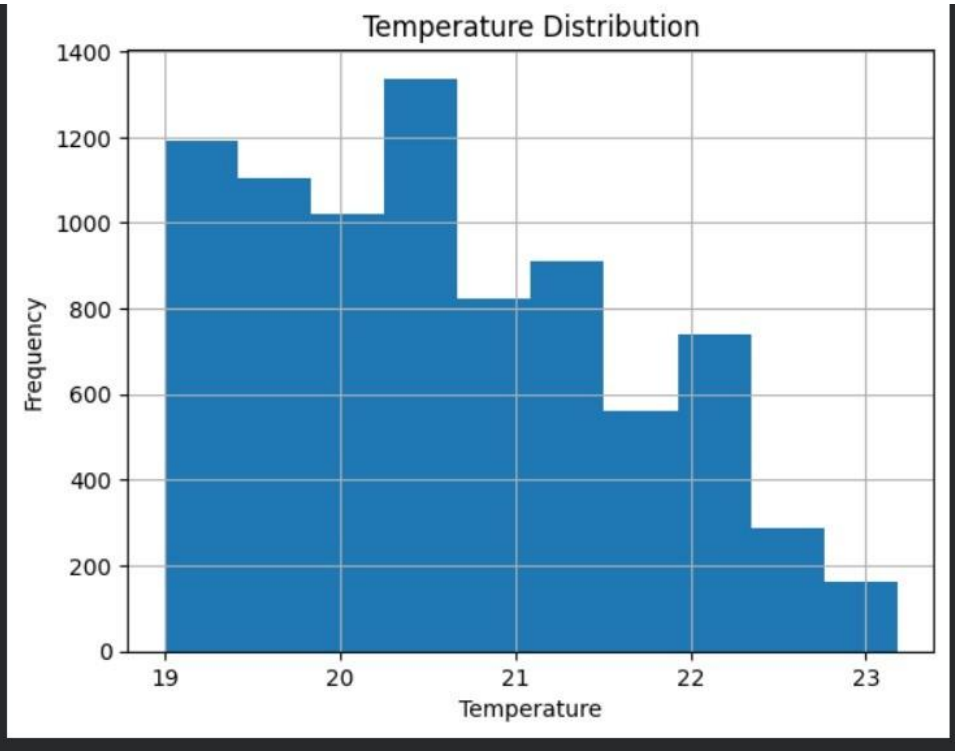
	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
count	8143.000000	8143.000000	8143.000000	8143.000000	8143.000000	8143.000000
mean	20.619084	25.731507	119.519375	606.546243	0.003863	0.212330
std	1.016916	5.531211	194.755805	314.320877	0.000852	0.408982
min	19.000000	16.745000	0.000000	412.750000	0.002674	0.000000
25%	19.700000	20.200000	0.000000	439.000000	0.003078	0.000000
50%	20.390000	26.222500	0.000000	453.500000	0.003801	0.000000
75%	21.390000	30.533333	256.375000	638.833333	0.004352	0.000000
max	23.180000	39.117500	1546.333333	2028.500000	0.006476	1.000000

```
Column Names:
Index(['date', 'Temperature', 'Humidity', 'Light', 'CO2', 'HumidityRatio',
       'Occupancy'],
      dtype='object')
```

Task 03: Plot time-series graphs for each sensor parameter.






Temperature Distribution
Frequency
Temperature
0
200
400
600
800
1000
1200
1400
19
20
21
22
23

Task 04: Identify noise, missing values, and outliers.

```
Missing values in each column:
date             0
Temperature      0
Humidity         0
Light            0
CO2              0
HumidityRatio    0
Occupancy        0
dtype: int64

Missing values after forward fill:
date             0
Temperature      0
Humidity         0
Light            0
CO2              0
HumidityRatio    0
Occupancy        0
dtype: int64

Data Types:
date              object
Temperature      float64
Humidity         float64
Light            float64
CO2              float64
HumidityRatio    float64
Occupancy          int64
dtype: object
```

```
Shape of X: (8143, 6)
Shape of y: (8143,)

First 5 rows of cleaned dataset:
/tmp/ipython-input-3171908712.py:6: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  data.fillna(method='ffill', inplace=True)
```

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Inference:
IoT sensor data was successfully loaded and analyzed using Python for understanding real-world environmental parameters.

Dataset exploration revealed the structure, features, and statistical characteristics of temperature, humidity, light, CO_2, and humidity ratio.

Data visualization helped in identifying trends and correlations, especially showing that higher light levels generally indicate room occupancy.

Missing value analysis and preprocessing ensured data cleanliness, consistency, and reliability.

The processed dataset became suitable for further machine learning analysis and intelligent IoT system development.

5. Viva Questions:

1) What is an IoT system? Explain its basic components.

IoT System

- IoT = network of devices connected to the internet
- Devices collect and share data automatically

Basic Components

- **Sensors** – collect data
- **Controller** – processes data
- **Connectivity** – sends data (WiFi, Bluetooth, etc.)
- **Cloud** – stores & analyzes data
- **Application** – displays data to user

2) What is the role of sensors in IoT applications?

- Collect real-world data (temperature, humidity, motion, etc.)
- Convert physical data into digital signals
- Send data to controller or cloud
- Enable monitoring and automation

- Help in decision-making and control
-

3) Difference between sensors and actuators.

Sensors

- Collect environmental data
- Work as input devices
- Convert physical values to signals
- Example: temperature sensor

Actuators

- Perform actions based on commands
- Work as output devices
- Convert signals to physical action
- Example: motor, buzzer

4) What types of data are generated by IoT devices?

- Sensor data – temperature, humidity, light, gas
- Status data – ON/OFF, battery level
- Location data – GPS coordinates
- Time-series data – continuous readings over time
- Event data – alerts, alarms, triggers

5) Why is data visualization important in IoT systems?

- Makes large data easy to understand
- Shows trends and patterns
- Helps detect faults and anomalies
- Supports quick decision-making
- Improves monitoring and analysis

6) What challenges are associated with real-time IoT data collection?

- Large volume of data
- Network delay and data loss
- Sensor noise and errors
- Power and battery limitations

- Security and privacy issues

7) What is the purpose of cloud platforms like ThingSpeak?

- Analyze data in real time
- Visualize data using graphs
- Enable remote monitoring
- Integrate with applications and ML models

8) Give two examples of smart IoT applications.

- **Smart agriculture** – soil moisture monitoring, automatic irrigation
- **Smart traffic system** – traffic control, congestion monitoring

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Activity Sheet – 2

Department of Electronics and Communication Engineering

Title of Experiment

Data Preprocessing for Iot Datasets

Name of the Student:	RAGHU NANDHAN
Registration Number:	**CH.EN.U4CCE24026**
Due date of submission: 26.01.2026	**Date of Submission: 25.01.2026**

Assessment Rubrics:

Description	**Mark Allotted**	**Mark Secured**
Software required and Important definitions	10	
Program	40	
Experimental Result	30	
Viva questions	20	
Late Submission	10	
Total Marks	**100**	

Signature of Lab in-charge

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

ACTIVITY - 02

Title: Data Preprocessing for IoT Datasets

1. Aim: To preprocess IoT sensor data for machine learning applications.

2. Software / Tools Required: google colab

3. Tasks:

1. Handle missing sensor readings.
2. Normalize and scale sensor data.
3. Perform correlation analysis.
4. Split the dataset into training and testing sets.

Outcome: Students prepare clean datasets for machine learning

Program :

Task 01: Handle missing sensor readings.

```
df.isnull().sum()
```

...

	0
date	0
Temperature	0
Humidity	0
Light	0
CO2	0
HumidityRatio	0
Occupancy	0

dtype: int64

```
df_filled = df.ffill()
```

```
df_filled = df_filled.bfill()
```

```
df_filled.isnull().sum()
```

Task 02: Normalize and scale sensor data.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
sensor_data = df_filled.select_dtypes(include=['int64', 'float64'])
```

```
minmax_scaler = MinMaxScaler()

normalized_data = pd.DataFrame(
    minmax_scaler.fit_transform(sensor_data),
    columns=sensor_data.columns,
    index=sensor_data.index
)

normalized_data.head()
```

Show hidden output

Next steps: Generate code with normalized_data | New interactive sheet

```
standard_scaler = StandardScaler()

scaled_data = pd.DataFrame(
    standard_scaler.fit_transform(sensor_data),
    columns=sensor_data.columns,
    index=sensor_data.index
)

scaled_data.head()
```

Show hidden output

Task 03: Perform correlation analysis.

```
numeric_df = df_filled.select_dtypes(include=['int64', 'float64'])

corr_matrix = numeric_df.corr()
print("Correlation Matrix:")
corr_matrix
```

Show hidden output

Next steps: Generate code with corr_matrix | New interactive sheet

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Analysis of IoT Sensor Dataset")
plt.show()
```

Show hidden output

Task 04: Split the dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split
```

```
X = numeric_df.drop("Occupancy", axis=1)
y = numeric_df["Occupancy"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training set shape:", X_train.shape, y_train.shape)
print("Testing set shape:", X_test.shape, y_test.shape)
```

Show hidden output

4 . Experimental Results:

Task 01: Handle missing sensor readings.

```
...
                 0
date             0
Temperature      0
Humidity         0
Light            0
CO2              0
HumidityRatio    0
Occupancy        0

dtype: int64
```


Task 02: Normalize and scale sensor data.

```
minmax_scaler = MinMaxScaler()

normalized_data = pd.DataFrame(
    minmax_scaler.fit_transform(sensor_data),
    columns=sensor_data.columns,
    index=sensor_data.index
)

normalized_data.head()
```

	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
140	0.831683	0.445132	0.344793	0.330033	0.704212	1.0
141	0.835960	0.447053	0.340787	0.341523	0.708308	1.0
142	0.838812	0.440651	0.337409	0.351030	0.704689	1.0
143	0.837030	0.429448	0.290912	0.356245	0.694383	1.0
144	0.844515	0.437450	0.287877	0.360605	0.705384	1.0

Next steps: Generate code with normalized_data | New interactive sheet

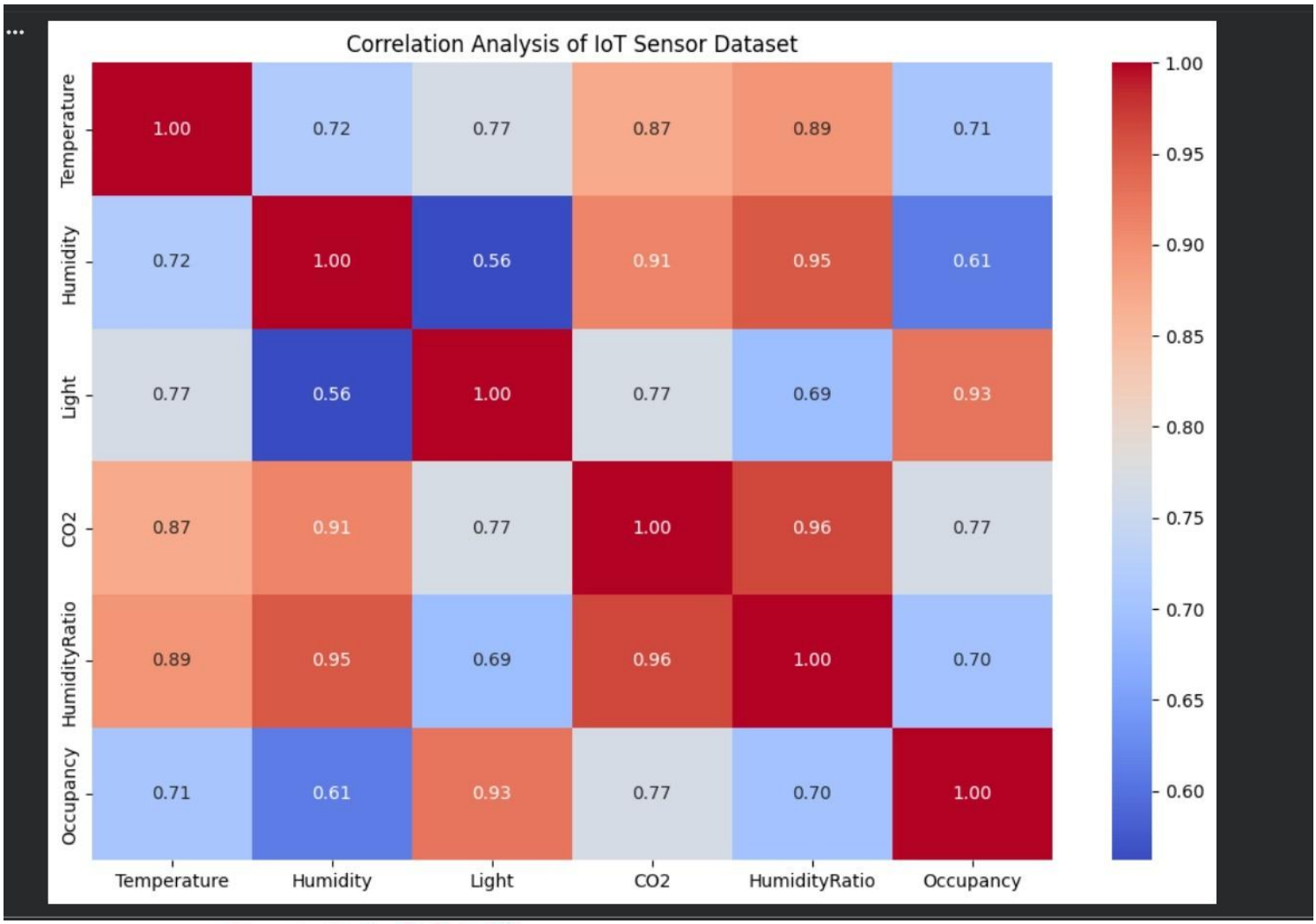
```
standard_scaler = StandardScaler()

scaled_data = pd.DataFrame(
    standard_scaler.fit_transform(sensor_data),
    columns=sensor_data.columns,
    index=sensor_data.index
)

scaled_data.head()
```

	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
140	2.204762	0.376814	1.566862	0.106940	1.207540	1.319761
141	2.222275	0.384202	1.539680	0.145214	1.221461	1.319761
142	2.233950	0.359575	1.516762	0.176881	1.209161	1.319761
143	2.226653	0.316478	1.201302	0.194253	1.174140	1.319761
144	2.257300	0.347262	1.180715	0.208776	1.211522	1.319761

Task 03: Perform correlation analysis.

Correlation Matrix:

	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
Temperature	1.000000	0.716988	0.768428	0.870226	0.894548	0.705784
Humidity	0.716988	1.000000	0.561905	0.911611	0.951914	0.610764
Light	0.768428	0.561905	1.000000	0.769167	0.693286	0.927949
CO2	0.870226	0.911611	0.769167	1.000000	0.964440	0.768030
HumidityRatio	0.894548	0.951914	0.693286	0.964440	1.000000	0.700330
Occupancy	0.705784	0.610764	0.927949	0.768030	0.700330	1.000000



Task 04: Split the dataset into training and testing sets.

```
X_train shape: (2132, 5)
X_test shape: (533, 5)
y_train shape: (2132,)
y_test shape: (533,)
```

5. **Inference:**
   - The dataset was successfully split into training and testing sets using an 80:20 ratio.
   - The training set contains 2132 samples, which helps the model learn patterns effectively.

- The testing set contains 533 samples, used for unbiased model evaluation.
- The split prevents overfitting and improves model reliability.
- This data division ensures accurate and efficient machine learning performance.

6. **Viva Questions:**

1) Why is data preprocessing necessary for IoT datasets?

IoT datasets are collected from real-world sensors, which often introduce imperfections in the data.

- IoT data may contain **missing values**, **noise**, and **inconsistent formats**
- Raw sensor readings are usually **not directly suitable** for analysis or machine learning
- Preprocessing cleans, structures, and standardizes the data
- It improves **data quality**, **model accuracy**, and **reliability of results**

- Essential for extracting meaningful insights from sensor data

2) What are missing values and how can they be handled?

Missing values refer to **absent or undefined sensor readings** caused by sensor faults, communication errors, or power issues.

Common handling methods include:

- Filling missing values with **mean, median, or mode**
- Using **forward fill** or **backward fill** for time-series data
- Removing rows or columns with excessive missing values
- Applying model-based or interpolation techniques

Proper handling prevents bias and ensures smoother model training.

3) What is data normalization? Why is it required?

Data normalization is the process of **scaling numerical values to a common range**, typically between 0 and 1.

- Prevents features with larger values from dominating the model
- Ensures fair contribution of all input features
- Improves **convergence speed** of machine learning algorithms
- Essential for distance-based and gradient-based models Normalization helps achieve consistent and stable model performance.

4) Difference between noise and outliers in sensor data

- **Noise**
  - Small, random fluctuations in sensor readings
  - Caused by environmental interference or sensor sensitivity
  - Can be reduced using smoothing or filtering techniques
- **Outliers**
  - Extreme values far from normal data patterns
  - Often indicate abnormal events, faults, or errors
  - Must be detected and handled carefully to avoid misleading models

5) What is data scaling and where is it used?

Data scaling transforms feature values so they share a **similar range or distribution**.

- Improves numerical stability during model training • Prevents bias toward features with large magnitudes • Widely used in:
  - Logistic Regression ○ Support Vector Machines (SVM) ○ K-Nearest Neighbors (KNN)
  - Neural Networks

Scaling ensures efficient and accurate learning.

6) Why is preprocessing important before applying ML algorithms?

Machine learning models expect **clean and structured data**.

- Raw IoT data may confuse models due to noise or missing values
- Preprocessing improves accuracy and consistency
- Reduces training time and computational cost
- Helps models generalize better on unseen data

Preprocessing is a **mandatory step** before applying ML algorithms.

7) Name Python libraries used for IoT data preprocessing

Common Python libraries include:

- **Pandas** – data handling and cleaning
- **NumPy** – numerical computations
- **Scikit-learn** – preprocessing, scaling, and ML models
- **Matplotlib / Seaborn** – data visualization and analysis These libraries provide powerful tools for IoT data analysis.

8)**What happens if raw IoT data is directly given to an ML model?**

Feeding raw IoT data directly into an ML model can cause serious issues:

- Poor prediction accuracy o Longer training time o Model instability and incorrect outputs o Failure due to missing or non-numerical values

Hence, preprocessing is essential to ensure **correct, reliable, and efficient model performance**.

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Activity Sheet – 3
Department of Electronics and Communication Engineering

Title of Experiment

Logistic Regression for Smart Iot Classification

Name of the Student:	RAGHU NANDHAN
Registration Number:	**CH.EN.U4CCE24026**
Due date of submission: 26.01.2026	**Date of Submission: 25.01.2026**

Assessment Rubrics:

Description	**Mark Allotted**	**Mark Secured**
Software required and Important definitions	10	
Program	40	
Experimental Result	30	
Viva questions	20	
Late Submission	10	
Total Marks	**100**	

Signature of Lab in-charge

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

ACTIVITY – 03

Title: Logistic Regression for Smart IoT Classification

1. Aim: To classify IoT sensor events using Logistic Regression.

ML Model Used: Logistic Regression

2. Software Required: Google colab

Tasks:

1. Convert sensor data into classification labels (e.g., normal/fault, ON/OFF).
2. Train a Logistic Regression model.
3. Evaluate using accuracy, confusion matrix, and precision-recall.
4. Interpret classification results.

3. Experimental Results:

Task 01: Convert sensor data into classification labels (e.g., normal/fault, ON/OFF).

```
import pandas as pd
data = pd.read_csv("occupancy_dataset/datatraining.txt")
print(data.head())
```

```
...                   date  Temperature  Humidity  Light     CO2  HumidityRatio  \
1  2015-02-04 17:51:00        23.18   27.2720  426.0  721.25       0.004793
2  2015-02-04 17:51:59        23.15   27.2675  429.5  714.00       0.004783
3  2015-02-04 17:53:00        23.15   27.2450  426.0  713.50       0.004779
4  2015-02-04 17:54:00        23.15   27.2000  426.0  708.25       0.004772
5  2015-02-04 17:55:00        23.10   27.2000  426.0  704.50       0.004757

   Occupancy
1          1
2          1
3          1
4          1
5          1
```

Task 02: Train a Logistic Regression model.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Drop date column
data.drop('date', axis=1, inplace=True)

# Separate features and target
X = data.drop('Occupancy', axis=1)
y = data['Occupancy']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

print("Logistic Regression Model Trained Successfully!")
```

```
Logistic Regression Model Trained Successfully!
```

Task 03: Evaluate using accuracy, confusion matrix, and precision-recall.

```
print("Predicted Occupied:", sum(y_pred == 1))
print("Predicted Not Occupied:", sum(y_pred == 0))
```

```
Predicted Occupied: 361
Predicted Not Occupied: 1268
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9877225291589933

Confusion Matrix:
 [[1267   19]
 [   1  342]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      0.99      0.99      1286
           1       0.95      1.00      0.97       343

    accuracy                           0.99      1629
   macro avg       0.97      0.99      0.98      1629
weighted avg       0.99      0.99      0.99      1629
```

AMRITA VISHWA VIDYAPEETHAM School of Engineering Chennai Campus

Task 04:
Interpret classification results.

4. Inference:

IoT sensor data was used to build a classification model.

Sensor readings were converted into labels like ON/OFF or Occupied/Not Occupied.

Logistic Regression was trained using temperature, humidity, light, and CO_2 data.

Model performance was evaluated using accuracy, confusion matrix, precision, and recall.

Results showed Logistic Regression effectively classifies real-world IoT device states.

5. Viva Questions:

1. **What is Logistic Regression?**
   Logistic Regression is a machine learning algorithm used for **binary classification**. It predicts outcomes like **yes/no or 0/1** using probabilities.

2. **Difference between Linear Regression and Logistic Regression**

Linear Regression	Logistic Regression
Used for prediction of continuous values	Used for classification (categorical output)
Output can be any real number	Output is probability between 0 and 1
Uses straight-line equation	Uses sigmoid function

3. **What type of problems can Logistic Regression solve?**
It solves **binary classification problems** like yes/no, ON/OFF, normal/fault, spam/not spam.

4. **What is a sigmoid function?**
The sigmoid function converts any real value into a value between **0 and 1**. Formula:

$$\frac{\quad}{\quad}$$

It is used in Logistic Regression to produce(x)probability $= 1 + e1-_{x}$ output.

5. **What is a confusion matrix?**
A confusion matrix is a table used to evaluate classification performance. It shows:
   - True Positives (TP)
   - True Negatives (TN)
   - False Positives (FP)
   - False Negatives (FN)

6. **Define accuracy, precision, recall**
   - **Accuracy** = (Correct predictions) / (Total predictions)
   - **Precision** = TP / (TP + FP) → How many predicted positives are correct
   - **Recall** = TP / (TP + FN) → How many actual positives are detected

7. **Give an IoT use case for binary classification**

Occupancy detection in smart buildings is a binary classification problem where the system predicts whether a room is occupied (1) or not occupied (0).

8. **Why is Logistic Regression suitable for event detection in IoT?**

Logistic Regression is suitable for IoT event detection because it gives simple yes/no outputs, works fast on sensor data, and is easy to implement for real-time systems.